CMDBUF RD RTL 设计说明

文档版本 1.2 2026 年 10 月 21 日

项目整体介绍

本项目设计的是 SoC Cross Bar 中的读命令缓冲模块 cmdbuf_rd。它连接上游最多三个 XAU 和下游存储访问逻辑。XAU 发出读请求后，命令先进入 cmdbuf_rd，再经过 Arbit 仲裁送到 MEM Bank；MEM 返回数据后，也由 cmdbuf_rd 整理并送回对应的 XAU。

设置这个模块的原因是多个 XAU 会竞争同一组存储资源。如果严格按照请求到达的先后顺序访问 MEM，排在前面的命令一旦碰到忙 Bank，后面的命令也会跟着等待，即使后面的命令要访问的 Bank 当前是空闲的。这会浪费可用的访问机会。

cmdbuf_rd 先把收到的命令保存起来，再从最早到达、尚未成功发出的五条命令中检查哪些目标可以访问。若有多个目标空闲，就选择其中最早到达的一条发送。因此，后来的命令可以先发往 MEM，但选择范围始终限制在这五条命令之内。

发往 MEM 的顺序改变后，数据完成的顺序也可能改变。为此，模块内部设置 OTB，专门保存返回的数据和奇偶校验结果。即使同一 XAU 的第三条命令先完成，数据也要暂存在 OTB 中，等第一条和第二条返回给该 XAU 后，才能返回第三条。

模块需要完成的工作

一次读请求从接收到响应结束，会同时占用 CMDBUF 中的一项命令记录和 OTB 中的一项数据记录。这两项使用相同的编号，合称一个 entry。模块共有九组 entry，由三个 XAU 共享；任意一个 XAU 最多占七组，并为其他 XAU 保留接收首条命令的空间。只有响应真正交给上游后，这组空间才能再次分配。

本设计允许命令接收、下游发射、MEM 返回和上游响应同时进行。每个环节每拍最多处理一条命令。设计的作用是在现有接口带宽内减少不必要的等待，实际能否连续完成访问，仍取决于 Bank 状态、仲裁结果和上游接收能力。

本文的范围

本文先介绍模块的整体结构和一条读请求的处理过程，再说明接口、内部实现以及具体时序场景。对应实现为 rtl/files.f 中的三份 SystemVerilog 文件。项目配套的仿真和形式验证用于检查命令是否丢失、重复、越序或返回错误数据；现有验证结果及其适用范围放在文档最后。

1 系统架构

从系统连接上看，cmdbuf_rd 位于 XAU 与存储仲裁逻辑之间。图中上半部分是命令向下游发送的路径，下半部分是数据返回的路径。内部的 CMDBUF 负责记录和调度命令，OTB 负责接住并保存 MEM 返回的数据。

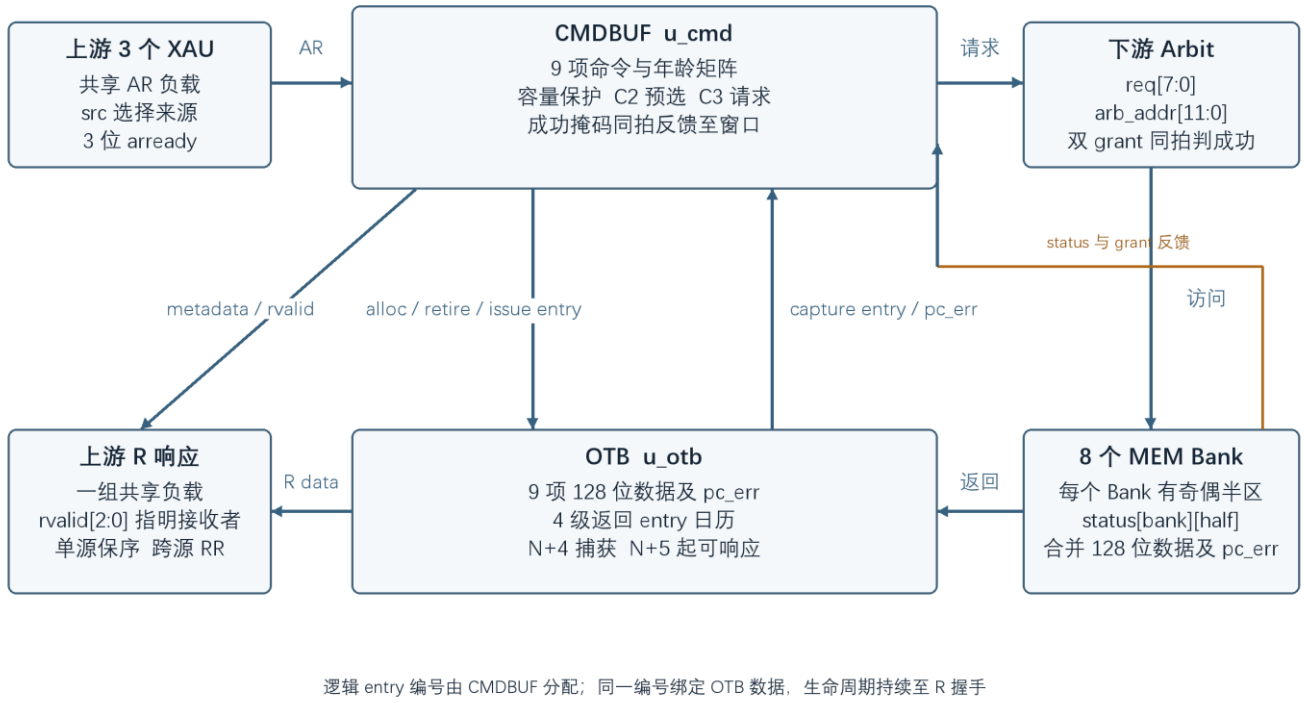


图 1 cmdbuf_rd 在读访问路径中的位置及内部连接

上游通过一组共享的 AR 信号提交命令，用 src 指明来自哪个 XAU。CMDBUF 接收命令时保存地址、ID、port_id 和地址检查结果，并同时在 OTB 中预留相同编号的空间。这样，MEM 返回数据时已经有确定的存放位置。

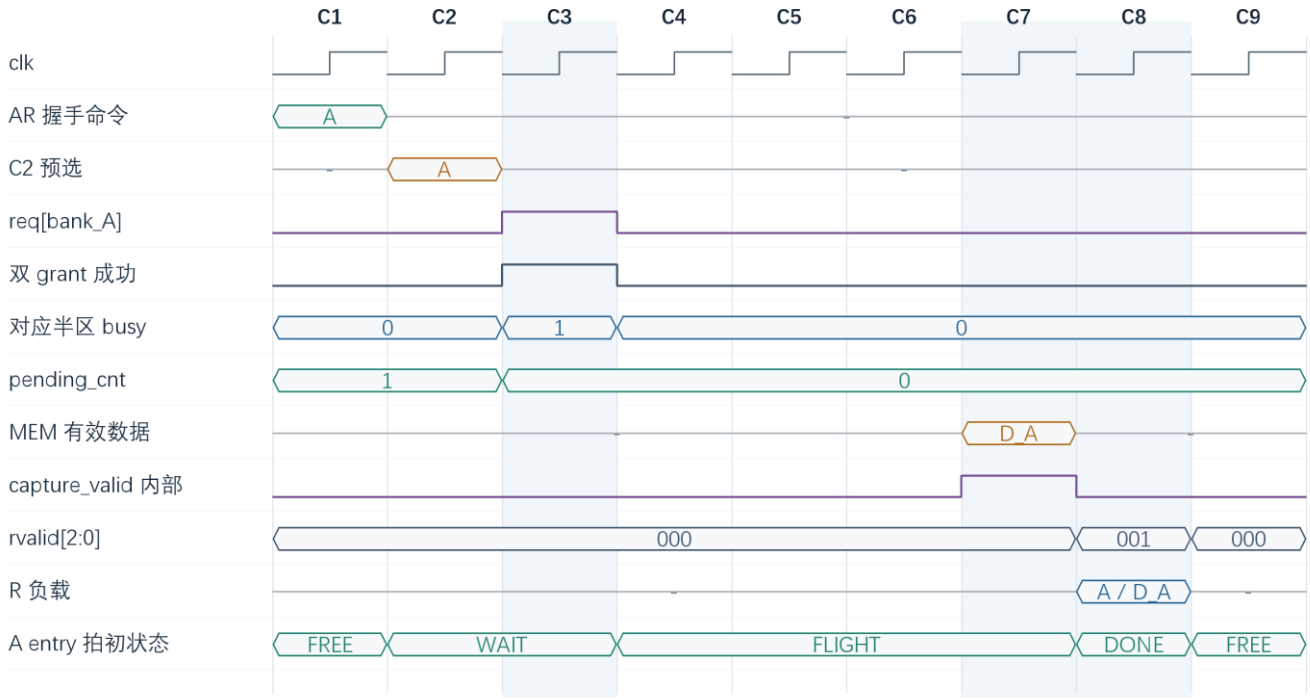
CMDBUF 根据 Bank 状态选择待发命令，向 Arbit 输出请求和地址。只有两个仲裁结果都表示本次读访问成功，CMDBUF 才将命令标记为已发出，并通知 OTB 开始等待返回。命令此时仍保存在 CMDBUF 中，供后续匹配响应和判断顺序使用。

OTB 在规定的返回时刻保存数据和 pc_err，并通知 CMDBUF 该命令已经完成。CMDBUF 先检查每个 XAU 最早的一条未响应命令，再从能够接收响应的 XAU 中轮询选择一个，通过共享的 R 通道返回结果。

三份 RTL 的分工是：cmdbuf_rd.sv 连接接口和子模块；cmdbuf_rd_cmd.sv 实现 CMDBUF 的管理与调度；cmdbuf_rd_otb.sv 实现返回计时和数据存储。

一条读请求的完整处理过程

先看只有一条命令 A 的情况。假设 A 来自 XAU0，目标 Bank 可以访问，仲裁第一次就成功，上游也一直能够接收响应。下图从 C1 接收命令开始，展示这条命令直到 C8 返回的全过程。



W01 协议示意：A 来自 XAU0，rready=111；busy 除成功拍外仅为本例取值

C1，上游请求被接收。CMDBUF 保存命令，同时占用一组 entry。C2，CMDBUF 检查目标 Bank 的奇偶半区是否空闲，选中 A，并把选择结果写入发射流水寄存器。C3，寄存器中的命令送到 Arbit；本拍两个仲裁结果都成功，A 才算真正发出。

从 C3 成功发射算起，MEM 固定延迟四拍返回，所以 C7 采样 A 的数据和 pc_err。OTB 在这个采样沿保存结果，C7 不能直接把这份新数据送到 R。最快到 C8，模块才向 XAU0 返回 A，并回收它占用的 entry。

有多条命令时，这些步骤可以重叠。例如 C3 发射 A 的同时，可以选择 B，也可以接收新的命令 C。各条命令仍按各自的阶段前进；仲裁失败、Bank 忙或上游暂时不能接收时，相关步骤会等待，后面的场景页将分别说明。

阅读导航

前面三页说明项目的用途、组成和基本流程。下面各章按接口、内部模块和具体场景展开。第一次阅读可以顺序看完模块设计；核对 RTL 或波形时，可以直接从目录跳到相应主题。

章节或场景	页码
项目整体介绍与系统架构	1 至 2
一条读请求的完整处理过程 场景 01	3
接口 位宽 握手与时序	5 至 8
CMDBUF 命令状态 容量 窗口与流水	9 至 14
OTB 返回计时及数据保存	15
单 XAU 保序与响应轮询	16
性能和实现要求	17
场景 02 至 03 连续输入与来源切换	18 至 19
场景 04 至 08 窗口 仲裁 重试与半区选择	20 至 24
场景 09 至 12 计数 数据保存 保序与轮询	25 至 28
场景 13 至 15 容量保护与同拍多事件	29 至 31
场景 16 至 18 ID 错误结果 grant 与复位	32 至 34
验证内容与已有结果	35
集成要求和文件入口	36

波形中的记号

C 和 T 是协议示意图的拍号；S 是已保存 VM 仿真中、复位释放后的采样拍号。图中每列表示一个时钟上升沿采样的值，寄存器值按更新前显示。组合输出 pending_cnt 则已经计入本拍接收和成功发射。

A、B 或 A1、A2、A3 是为了说明顺序而给命令起的代号，不是 arid，也不等于物理 entry 编号。图中的短横线表示这一拍没有有效事务或不需要关注该值。图注注明了波形来自 VM 记录还是协议示意。

2 接口与协议

AR 接收通道

三个 XAU 共用一组 AR 命令负载，因此每拍最多接收一条。src 指明当前命令的来源，aready 的三个位分别告诉三个 XAU 是否还有接收容量。下表的输入和输出方向均以 cmdbuf_rd 为参照。

端口	方向	位宽	说明
clk	输入	1	工作时钟，正常事务在上升沿采样。
rst_n	输入	1	低有效异步复位。
arvalid	输入	1	当前 AR 命令有效。aready 全为 0 时，上游必须将它置 0。
arready	输出	[2:0]	bit x 表示可以接收 XAUx 的命令。
araddr	输入	[17:0]	完整读地址，各字段的下游用途见 Arbit 与 MEM 接口。
src	输入	[1:0]	有效命令只允许取 0、1、2，分别对应三个 XAU。
arid	输入	[7:0]	命令 ID，随响应原样返回，允许不同命令使用相同值。
ar_port_id	输入	[4:0]	随响应原样返回的 port_id。
arresp	输入	[1:0]	00 表示地址正常，11 表示地址越界。越界命令仍发送给 MEM。

何时算接收成功

```
ar_fire = arvalid && (src < 3) && arready[src];
```

只有 ar_fire 为 1，模块才保存这条命令并分配 entry。例如 src=0、aready=110 时，即使 arvalid=1，也不能接收这条 XAU0 命令。其他 XAU 的 ready 为 1，不会改变当前命令的来源。

上游可以切换 src，向仍然 ready 的来源提供命令。当前约定只要求 aready 全零时 arvalid 为零，没有额外要求未接收命令的负载保持不变，也没有约定上游选择三个来源的公平性。这是一套项目自定义的 AR 协议。

R 响应通道与错误状态

R 通道共用一组数据、地址和标识字段。模块通过 rvalid 的置位位置选择接收这组负载的 XAU。一个采样拍最多只能有一位 rvalid 为 1。

端口	方向	位宽	说明
rvalid	输出	[2:0]	为 000 或只有一位为 1；置位 bit 指明响应对象。
rready	输入	[2:0]	各 XAU 是否能够在本拍接收响应。
rdata	输出	[127:0]	该命令此前从 MEM 捕获并保存在 OTB 中的数据。
rid	输出	[7:0]	该命令的 arid。
r_port_id	输出	[4:0]	该命令的 ar_port_id。
raddr	输出	[2:0]	该命令的 araddr[2:0]。
rresp	输出	[1:0]	该命令最终的错误状态。

只有 rready[x]=1，模块才可能拉高 rvalid[x]。当某个 XAU 不能接收时，它不参加本拍的响应轮询，数据继续保留在 OTB 中。若三个 rready 都为 0，rvalid 也必须为 000。

一次有效 R 响应同时交付同一条命令的 raddr、rid、r_port_id、rdata 和 rresp，并回收这条命令的 entry。不能先返回标识、再单独返回数据，也不能把不同命令的字段拼成一次响应。

错误状态如何形成

接收时的 arresp	MEM 返回的 pc_err	返回的 rresp
00	0	00 正常
00	1	01 奇偶校验异常
11	0	11 地址越界
11	1	11 地址越界

地址越界的优先级高于奇偶校验异常。响应时使用 entry 中保存的地址检查结果和 pc_err，不能读取当前 MEM 总线上的 pc_err 来决定旧命令的状态。当 rvalid=000 时，R 负载没有协议意义，验证也不检查其取值。

Arbit 与 MEM 接口

发往下游的命令用 req 选择 Bank，用 arb_addr 传递行地址和奇偶半区。MEM 返回数据只有合并后的一组端口，没有独立的返回有效信号，也没有返回命令编号。

端口	方向	位宽或形状	说明
req	输出	[7:0]	req[b]=1 表示本拍请求 Bank b；最多置位一位。
arb_addr	输出	[11:0]	原命令 araddr[17:6]，共 12 位。
pending_cnt	输出	[3:0]	当前尚未成功发出的命令数，为组合输出。
ar_rd_gnt	输入	[7:0]	Arbit 对本拍读请求的仲裁结果。
rw_rd_gnt	输入	[7:0]	本拍读写冲裁结果，置位表示选择读。
rd_mem_status	输入	8 组 2 bit	每组对应一个 Bank，低位表示偶半区，高位表示奇半区。
mem_rdata	输入	[127:0]	固定返回时刻提供的一组读数据。
pc_err	输入	1	与该组数据同时提供的奇偶校验异常。

地址和状态的对应关系

```
input wire [1:0] rd_mem_status [7:0];
```

原地址字段	作用
araddr[17:7]	行地址，送到 arb_addr[11:1]。
araddr[6]	选择偶半区或奇半区，送到 arb_addr[0]。
araddr[5:3]	选择 Bank0 至 Bank7，译码成 req。
araddr[2:0]	保留到响应时作为 raddr 返回。

rd_mem_status[7]=2'b00 表示 Bank7 两个半区都空闲；01 表示偶半区忙，10 表示奇半区忙，11 表示都忙。用地址 [5:3] 找 Bank，[6] 选其中一位。本项目 RTL 禁用二态 bit；顶层、子模块和验证接口均用四态 wire，保留 X/Z，避免未知状态被转换为空闲。

采样时序与下游握手

本文以时钟上升沿作为拍的采样点。组合判断在采样沿前形成，寄存器在采样沿后更新。例如 C2 完成的预选在该沿写入寄存器，C3 才对外表现为请求。新 AR 在接收当拍不能直接跳到预选结果。

命令发送分为三个阶段

阶段	本条命令的处理	可以同时进行的其他工作
C1 接收	保存 AR 命令并分配 entry。	选择较早命令，或发射已经选中的命令。
C2 选择	在五项窗口内检查空闲状态，选出最老候选。	继续接收新 AR，处理当前 req 的仲裁结果。
C3 请求	输出登记的 req 和地址，获得同拍仲裁结果。	重新计算窗口，选择下一拍要请求的命令。

真正成功的条件

```
issue_success = |(req & ar_rd_gnt & rw_rd_gnt);
```

三个向量必须在同一个 Bank 位上同时为 1。例如 req[3]=1 时，只有 ar_rd_gnt[3] 和 rw_rd_gnt[3] 都为 1，才能认定当前请求成功。其他 Bank 的 grant 不能使这条命令成功。

```
(req[i] && ar_rd_gnt[i] && rw_rd_gnt[i])
    |-> rd_mem_status[i][arb_addr[0]];
```

系统还约定：请求成功的当拍，目标半区的状态必须为忙。因此 C2 选择时看到空闲、C3 成功时看到忙，是正常的时序。C3 的 req 已由上一拍登记，不能再因为本拍状态变忙而撤销。状态为忙本身也不能用来判断仲裁成功。

MEM 返回与 R 响应

若在 N 拍成功发射，就在 N+4 拍采样 MEM 数据和 pc_err。它们只在这一个返回拍属于该命令，模块必须及时保存。最快到 N+5 才能通过 R 响应；若该命令前面还有同源旧命令，或上游不能接收，则继续保存在 OTB 中等待。

3 CMDBUF 设计

CMDBUF 保存九条尚未结束的命令，并决定何时接收、何时发射、何时响应。这里的“尚未结束”包括等待发射、已经发射但等待 MEM，以及数据已经返回但还没有交给 XAU 三种情况。

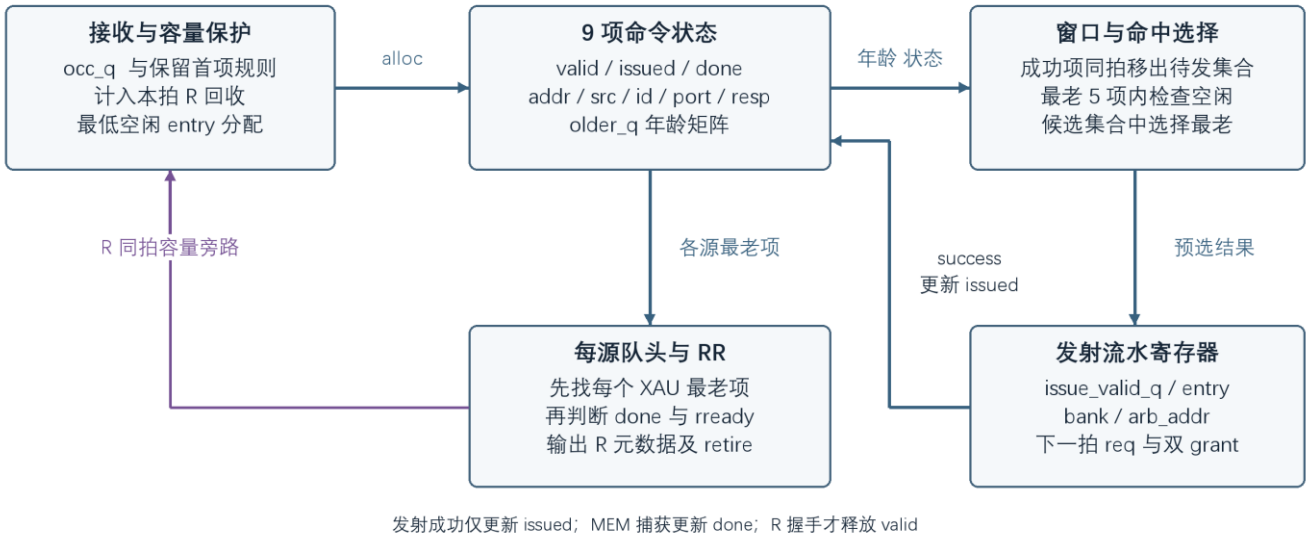


图 2 CMDBUF 内部的命令管理和调度关系

命令记录中的地址用于选择 Bank，来源用于区分 XAU，ID 和 port_id 用于形成响应。每条记录还保存已发射、已完成等状态。命令之间的先后关系由年龄矩阵保存，它同时服务于下游的最老候选选择和上游的单 XAU 保序。

寄存器	数量或位数	用途
valid_q / issued_q / done_q	各 9 bit	分别表示 entry 被占用、已成功发射、已收到数据。
addr_q / src_q	9×18 / 9×2 bit	原始地址和来源。
id_q / port_q / arresp_q	9×8 / 9×5 / 9×2 bit	响应标识和接收时的地址错误状态。
older_q	9×9 bit	记录任意两条有效命令的接收先后。
occ_q / pending_q	3×4 / 4 bit	各 XAU 占用数和寄存的待发命令数。
issue_* / rr_ptr_q	20 / 2 bit	下一次请求的信息和响应轮询起点。

命令状态与 entry 回收

一条命令被接收后，会先等待发射，再等待 MEM 返回，最后等待上游响应。下图用四个状态描述这个过程。RTL 实际使用 valid、issued、done 三个标志位表示状态，没有另设一个四状态的枚举寄存器。

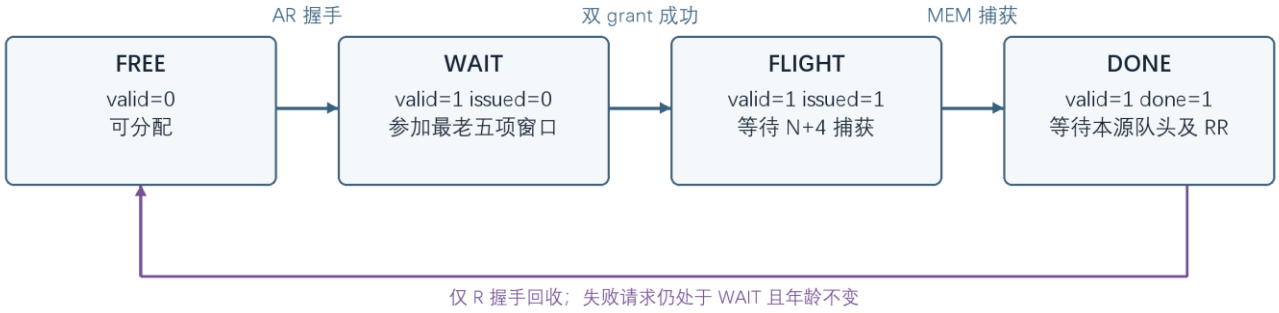


图 3 一组 entry 从分配到回收的过程

状态	valid issued done	何时进入下一状态
FREE 空闲	000	AR 接收成功，保存新命令后进入 WAIT。
WAIT 待发	100	下游仲裁成功后进入 FLIGHT；失败则继续等待。
FLIGHT 等待数据	110	在成功后的第四拍捕获数据，进入 DONE。
DONE 等待响应	111	R 响应成功后回到 FREE，或被同拍新 AR 直接复用。

发射成功只改变 issued，不能释放 entry。MEM 返回只使数据准备完成，也不能释放 entry。这两个阶段仍然需要保存命令身份和先后顺序。只有上游已经收到完整响应，旧命令的记录才可以清除。

如果旧命令在本拍完成 R 响应，同一拍又有新 AR 接收并复用相同编号，那么该编号在采样沿后直接属于新命令。新命令的 issued 和 done 必须清零，年龄也要重新变成最年轻，不能继承旧命令的状态。

共享容量与三个来源的保护

九组 entry 可以动态共享，但要保证某个 XAU 长时间积压时，不会把另外两个 XAU 的接收空间全部占用。具体做法是为每个来源保留首条命令的容量，剩余六组由各来源共同使用。这样单个 XAU 最多可以占用七组。

判断新命令能否接收时，先考虑本拍 R 已经释放的空间，再检查新命令是否会侵占其他来源的首条容量。因此即使拍初九组全满，只要本拍有合适的 R 回收，仍可能同时接收新 AR。

三个来源的占用	本拍 R 回收	aready[2:0]	原因
0 / 0 / 0	无	111	任何来源都可以提交首条命令。
7 / 0 / 0	无	110	XAU0 已用完共享部分，保留另两源各一组。
4 / 4 / 0	无	100	六组共享容量已用完，最后一组属于 XAU2 的首条容量。
7 / 1 / 1	无	000	九组全满，当前不能接收。
7 / 1 / 1	XAU0	111	释放了一组共享容量，三个来源均可使用。
7 / 1 / 1	XAU1	010	只恢复 XAU1 的首条容量，其他来源不能占用。

RTL 中的计算方式

```
occ_after[x] = occ_q[x] - retire_this_source[x];
free_after = 9 - sum(occ_after[x]);
shared_after = sum(max(occ_after[x] - 1, 0));
aready[x] = rst_n && (free_after > 0) &&
            ((occ_after[x] == 0) || (shared_after < 6));
```

收到 AR 后，从空闲项和本拍已回收项中选择编号最小的一项。仅检查“每源少于七项”是不够的：5 / 4 / 0 会把九组全占满，XAU2 无法再提交首条命令。这种分配不符合容量保护要求。

命令年龄与五项乱序窗口

命令的年龄由 AR 实际接收顺序决定，先接收的更老。ID 大小、物理 entry 编号和发射次数都不决定年龄。RTL 用 older_q[i][j]=1 表示 entry j 中的命令比 entry i 中的命令更早接收。

新命令分配到 entry i 时，所有仍然有效的旧命令都比它老；旧命令之间的关系保持不变。发射失败、发射成功和 MEM 返回都不会改变已有命令的相对年龄。回收旧命令时才清掉它的关系，新命令复用这个编号后重新建立关系。

先确定窗口 再检查空闲 最后比较年龄

每拍先找出所有尚未成功发射的命令，再按年龄取最老五条，形成乱序窗口。只有这五条能检查目标半区是否空闲。在通过空闲检查的候选中，选择年龄最老的一条，并把结果登记为下一拍请求。

```
rank_pool = valid_q & ~issued_q & ~success_mask;
rank[i] = popcount(older_q[i] & rank_pool);
window[i] = rank_pool[i] && (rank[i] < 5);
idle[i] = !rd_mem_status[addr_q[i][5:3]][addr_q[i][6]];
candidate = window & idle;
selected[i] = candidate[i] &&
              ((older_q[i] & candidate) == 0);
```

成功和失败怎样影响窗口

假设窗口按年龄排列为 A、B、C、D、E，F 排在窗口外。若本拍 C 发射成功，就在本拍的预选计算中排除 C，窗口随即成为 A、B、D、E、F。F 可以立即参与这次预选，并在下一拍发出请求。success_mask 就用于实现这一点。

若 C 仲裁失败，它仍属于原窗口，年龄也不变。如果本拍检查后它仍然是最老的空闲候选，就可以再次选中，下一拍继续请求；若它的目标已经变忙，则可以改选窗口内其他命令。

如果 A 至 E 的目标全部忙，即使 F 的目标空闲也不能发送 F。窗口必须等待内部某条命令成功发出后，才腾出位置给 F。窗口还没有填满时，新接收命令可以从下一拍开始补入。

发射流水与仲裁结果

发射端只有一组流水寄存器，用来保存本拍需要请求的命令。它每拍都可以装入新的预选结果。下面的两个状态表示这组寄存器是否有效，不表示整个模块一次只能处理一条命令。

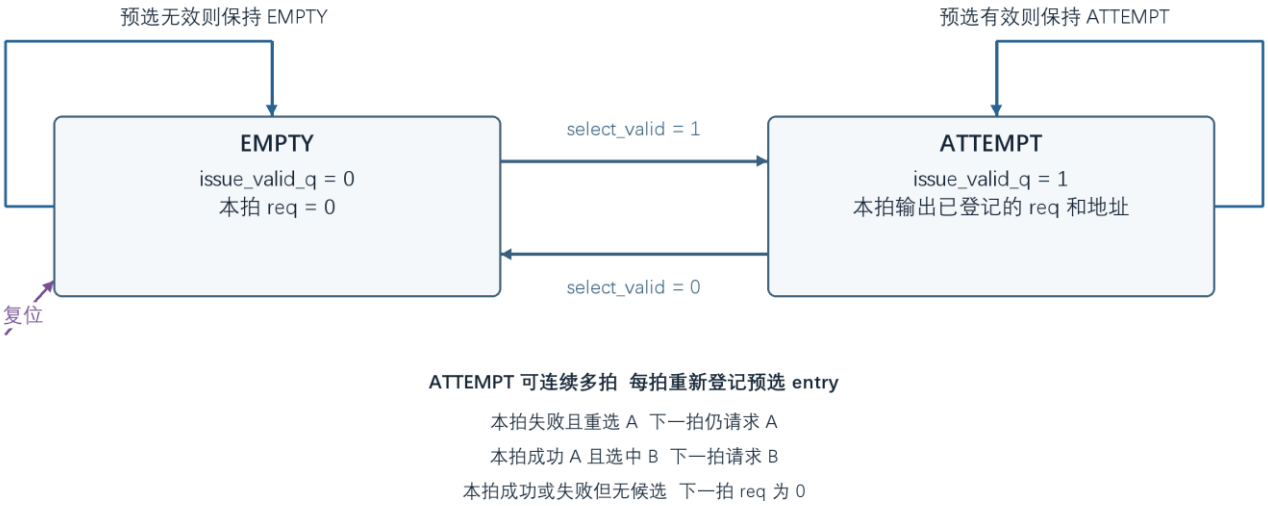


图 4 发射流水寄存器的两种状态

EMPTY 表示 issue_valid_q=0，本拍没有请求；ATTEMPT 表示 issue_valid_q=1，本拍按照已保存的 Bank 和地址发起请求。下一拍进入哪个状态，由本拍是否选出下一条命令决定。

本拍正在做什么	本拍预选结果	下一拍的请求
没有请求	未选中命令	仍无请求。
没有请求	选中 B	请求 B。
请求 A 失败	再次选中 A	继续请求 A，不插入固定空拍。
请求 A 失败	改选 B	请求 B，A 留在待发集合中。
请求 A 成功	选中 B	请求 B，A 不再参加发射选择。
请求 A 成功或失败	未选中命令	下一拍无请求。

流水寄存器保存一个有效位、4 位 entry 编号、3 位 Bank 编号和 12 位地址。仲裁失败后并不会简单锁住这些寄存器；下一次尝试仍来自本拍重新执行的选择。只有仲裁成功，OTB 才记录一次后续返回。

待发计数与同拍更新

pending_cnt 告诉下游还有多少条命令尚未成功发射。它与 entry 占用数不同：命令发射成功后，待发数减少，但 entry 仍然被占用，直到 R 响应结束才释放。

```
pending_cnt = pending_q + ar_fire - issue_success;
pending_q_next = pending_cnt;
```

pending_cnt 是组合输出，必须反映本拍的两个事件。若拍初有五条待发命令，本拍成功一条且没有新 AR，输出在本拍就是四；若同时又接收一条 AR，输出仍为五。MEM 返回和 R 回收本身不直接减待发数。

四种事件可以同时发生

事件	更新内容
AR 接收	保存新命令、分配 entry、更新年龄和来源占用。
发射成功	将命令置为已发射，待发数减一，OTB 记录返回时刻。
MEM 捕获	保存对应 entry 的数据和 pc_err，并置完成标志。
R 响应	交付旧命令的结果，回收 entry，更新轮询指针。

这些事件通常作用于不同命令，应分别执行。不能用一组全局 if / else 把它们处理成互斥关系。AR 和 R 可以针对同一个物理编号，但含义是旧命令回收后由新命令接管。

同一 entry 的更新先后

```
valid_next = (valid_q & ~retire_mask) | alloc_mask;
occ_next[x] = occ_q[x] + ar_for_x[x] - r_for_x[x];
```

对新分配和旧回收同时命中的 entry，新分配必须生效：写入新命令，清除旧数据，并把 issued、done 置零。正常生命周期中，正在捕获 MEM 的命令不允许同拍通过 R 返回，所以它不会与自己的回收发生冲突。

4 OTB 设计

OTB 为每条已接收命令保留一份数据空间，并在 MEM 返回时把数据存入对应 entry。因为 MEM 端口没有返回编号，OTB 必须根据“哪条命令在四拍前成功”来判断本拍数据属于谁。

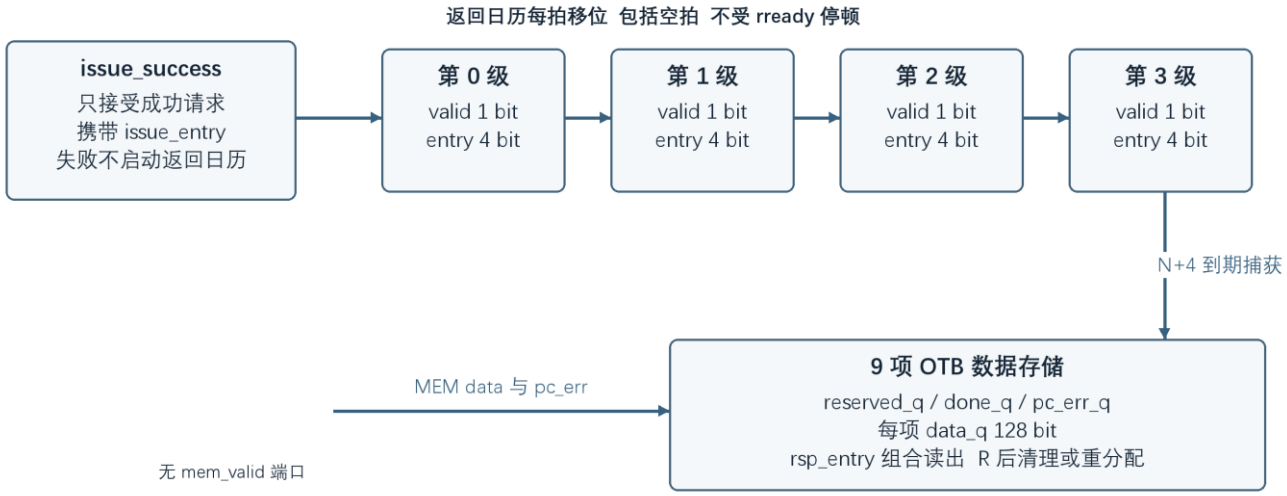


图 5 OTB 的返回计时流水和数据存储

OTB 用四级移位寄存器记录成功事件和 entry 编号，图中称为返回日历。若 N 拍成功，则 N+1 拍初记录在第 0 级，随后每拍前移，到 N+4 拍初出现在第 3 级。OTB 就在 N+4 的采样沿保存 mem_rdata 和 pc_err。

保存内容	规模	作用
ret_valid_q / ret_entry_q	4 bit / 4×4 bit	记录过去四拍的成功事件及对应 entry。
data_q	9×128 bit	保存每条命令的返回数据。
pc_err_q	9 bit	与数据同时保存的奇偶校验异常。
reserved_q / done_q	各 9 bit	记录预留和完成状态，用于内部一致性检查。

没有成功请求的拍写入一个无效位置，计时流水仍继续前移，不能因上游未就绪而暂停。MEM 捕获后，OTB 用 capture_valid 和 capture_entry 通知 CMDBUF；数据从下一拍起才具备响应资格。即使 rready 全零，OTB 也必须接住这次一拍宽的返回，并保存到 R 回收为止。

5 响应保序与轮询

响应选择先解决同一 XAU 的顺序，再解决三个 XAU 之间谁先使用 R 通道。对每个 XAU，先找它最早接收且尚未响应的命令，称为队头。只有队头的数据已经返回，而且该 XAU 的 rready 为 1，才可以参加本拍轮询。

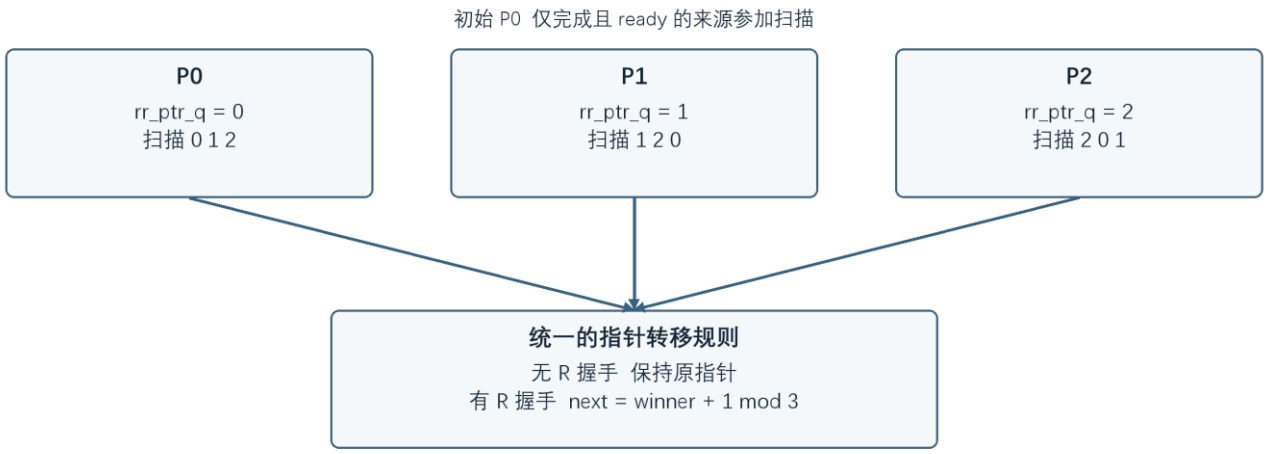


图 6 响应轮询的扫描起点和更新规则

例如 XAU0 按顺序提交 A1、A2、A3，A3 先完成时也不能直接成为响应候选。XAU0 的队头仍是 A1；必须依次返回 A1、A2 后，才轮到 A3。其他 XAU 的已完成队头不受这次等待影响，可以继续使用 R 通道。

本拍响应结果	下一拍扫描起点
没有响应	保持原指针。
返回给 XAU0	从 XAU1 开始。
返回给 XAU1	从 XAU2 开始。
返回给 XAU2	从 XAU0 开始。

复位后从 XAU0 开始扫描，按 0、1、2 循环检查，选中第一个满足条件的来源。有实际响应才更新指针，没有响应则保持。若某个来源的队头始终完成且一直 ready，它最多等待另外两个来源各响应一次；这个保证不包含等待 Bank 空闲或等待上游提交命令的时间。

6 性能与实现要求

当窗口中持续存在可访问目标，仲裁连续成功，并且各来源能够及时接收结果时，模块可以达到每拍接收一条、发射一条、捕获一条并响应一条。这里说的是各阶段重叠后的吞吐率，不是一条命令能在一拍内走完整个流程。

会产生等待的情况

条件	模块的行为
窗口中五个目标都忙	暂停发射，窗口外的空闲目标不能越过窗口。
连续命令访问同一半区	成功拍该半区为忙，不能同拍再选下一条同半区命令。
同源后续命令先完成	OTB 保存数据，等待该来源更早的命令先响应。
上游长期不 ready	已完成数据继续占用 entry，容量不足时停止接收。
下游长期不给成功 grant	命令留在待发集合中，保持年龄并按规则重试。

对于同一半区，即使成功后的下一拍就重新空闲，下一次成功也至少间隔一拍。若同一 Bank 的另一个半区空闲，且窗口中存在访问它的命令，则可以继续预选。两个半区必须分别判断。

组合路径和存储规模

同拍补入窗口需要把当前成功结果反馈到窗口和最老选择逻辑；满池同进同出需要把 R 回收结果反馈到 arready。这两条组合路径是既定时序的一部分。pending_cnt 又依赖本拍 AR 和 grant，集成时应检查上下游是否因此形成组合环。

OTB 仅数据部分为 9×128=1152 bit，命令年龄矩阵为 9×9=81 bit，此外还有元数据和控制寄存器。实际面积与最高频率需要综合及静态时序分析确定。本版没有给出目标工艺或频率，也不支持通过只修改一个常量就任意改变容量、窗口或返回延迟。

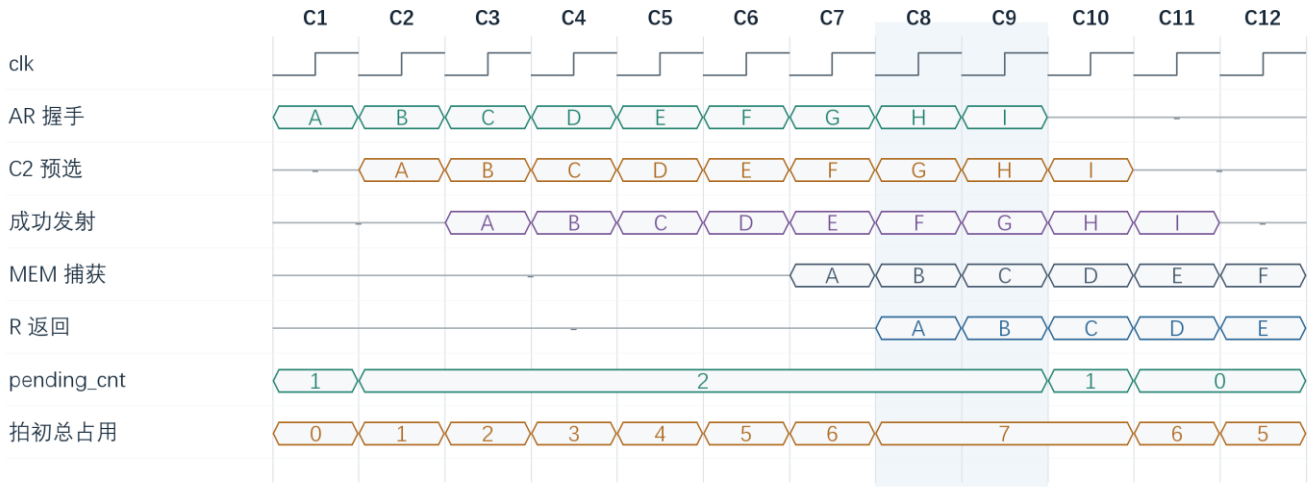
复位和适用范围

低有效复位清空命令状态、计数、年龄关系、发射流水和 OTB，响应指针回到 XAU0。运行中若 MEM 还保留旧请求，系统必须同时清除或隔离旧返回。当前接口按单时钟、每拍单命令、固定四拍返回设计；可变延迟、突发读和跨时钟传输需要另行定义协议。

7 场景与波形

场景 02 连续请求

这个场景用于观察流水怎样重叠。命令 A 至 I 连续从同一 XAU 进入，目标安排为可以连续访问的不同半区，每次下游仲裁都成功。



W02 协议示意：同源连续流，不同可用半区；C8 起接收和回收同时发生

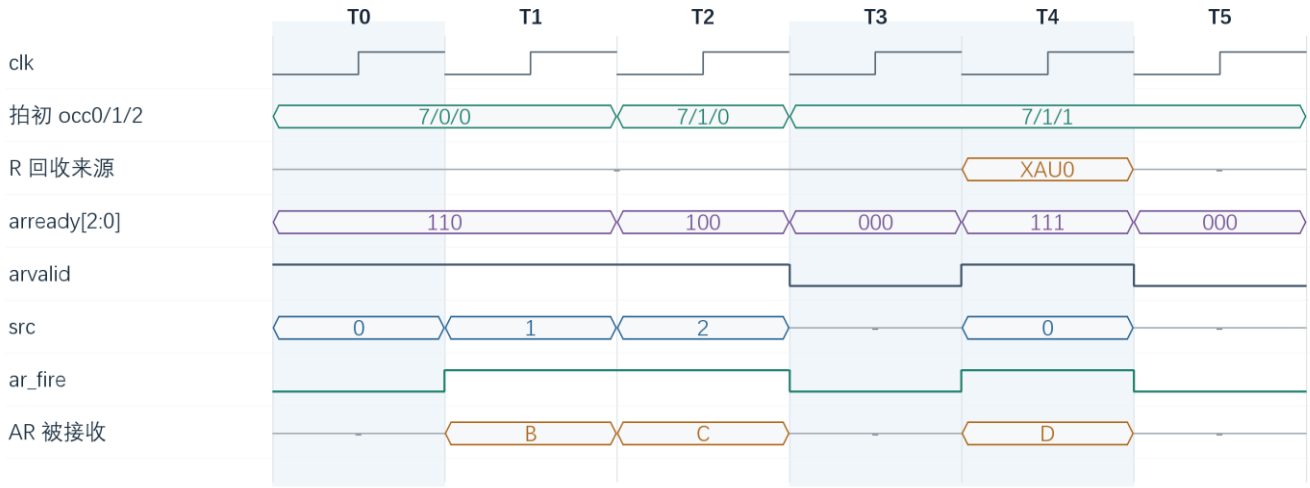
C3 发射 A 时，选择逻辑正在选择 B，上游同时提交 C。到 C8，模块同时接收 H、发射 F、捕获 B 的 MEM 数据，并向上游返回 A。四件事各自处理不同命令，可以在同一拍完成。

C8 接收 H 之前，先计入本拍 A 响应释放的容量，因此同一 XAU 的占用仍不超过七项。这里的“先计入”是组合计算关系，不表示把一个时钟周期拆成两次独立握手。

C9 之后停止输入，模块仍继续发出已经保存的 H、I。待发计数降到零时，还有命令在等待数据或等待响应，所以 pending_cnt=0 只能表示没有待发命令，不能表示全部事务都已结束。

场景 03 某个来源不能接收时切换来源

假设 XAU0 已经占用七组 entry，另外两个 XAU 还没有占用。模块应阻止 XAU0 继续增加占用，同时允许其他来源提交首条命令。



W03 协议示意：起始已有七条 XAU0 命令；T4 有已完成队头可响应

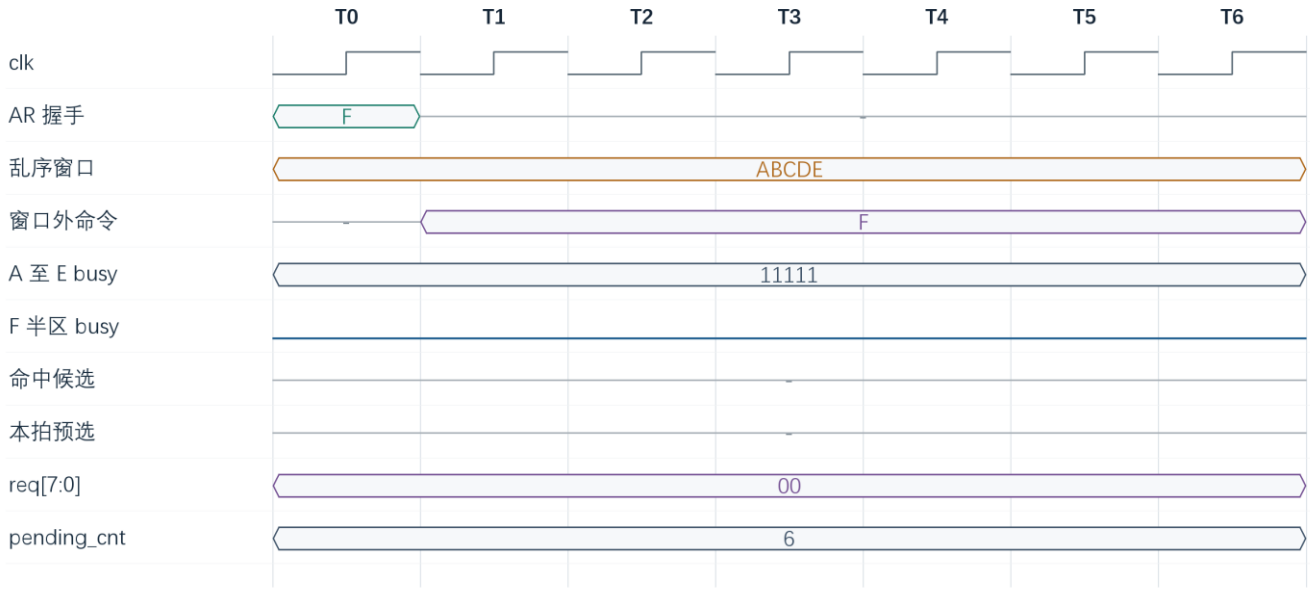
T0 的 arready=110，但上游仍给出 src=0，所以本拍不接收这条命令。T1 上游改为 XAU1，命令 B 被接收；T2 又接收 XAU2 的命令 C，占用成为 7 / 1 / 1。

T3 没有可分配空间，arready=000，按输入约定 arvalid 也必须为 0。T4 已完成的 XAU0 队头通过 R 返回，释放一组共享容量，本拍 arready 恢复为 111，可以同时接收 XAU0 的新命令 D。

被拒绝的 AR 不分配 entry，也不改变年龄和待发计数。多个 ready 位为 1 仅表示上游可以选择这些来源之一，实际共享 AR 仍然每拍只接收一条。图中的旧队头在 T0 之前已经完成 MEM 返回。

场景 04 五项窗口都忙而第六项空闲

窗口里已经有 A 至 E 五条待发命令，它们的目标半区都忙。此时又接收到 F，F 的目标是空闲的。这个场景说明窗口范围怎样限制发射。



W04 协议示意：满窗口的五条命令均忙，F 的半区始终空闲

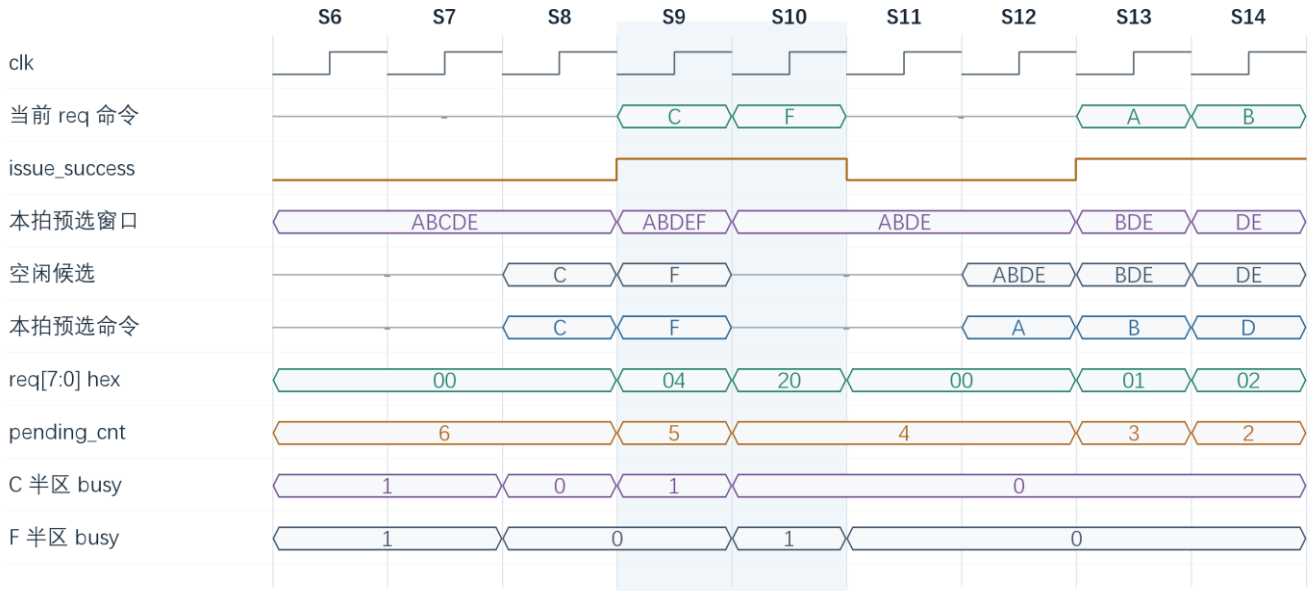
T0 接收 F 后，pending_cnt 当拍增加到六。F 从 T1 起成为已保存命令，但仍按年龄排在第六，不能进入五项窗口。因此图中的预选和 req 一直无效。

必须等窗口内某条命令的目标变为空闲，选中并成功发出后，F 才能补入窗口。仅增加新命令，或者让之前已经发出的其他命令完成响应，都不能使 F 越过 A 至 E。

如果 rd_mem_status 一直保持图中的状态，模块就一直等待。这符合五项窗口的设计要求。验证该规则时，需要检查发送的时间和地址，而不只是最终返回的数据是否正确。

场景 05 窗口中间的命令成功发射

A 至 F 按顺序到达，起初只有窗口中的 C 可以访问。C 发射成功后，F 应在同一拍补进窗口，参加下一次选择。



W05 VM VCD 22_nonhead_refill

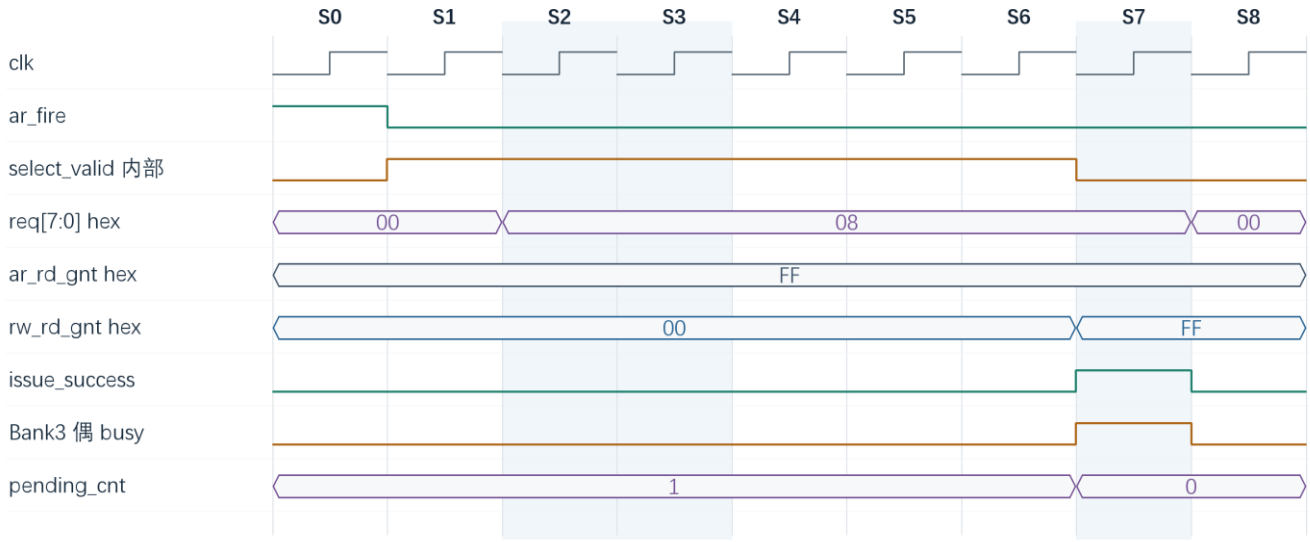
S8 的窗口是 A、B、C、D、E，选择逻辑选中 C。S9 请求 C 并获得成功结果，本拍预选时就排除 C，窗口变为 A、B、D、E、F。F 因而在 S9 被选中，S10 发出请求。

A、B、D、E 仍保持原来的先后关系，不能因为 C 成功就一起移除。S12 它们的目标都空闲后，首先选择最老的 A。S13 发射 A 成功的同时，又可以选择 B。

采样拍	发射与计数	本拍选择
S9	C 成功，待发数由 6 变为 5。	F 同拍进入窗口并被选中。
S10	F 成功，待发数由 5 变为 4。	剩余目标仍忙，没有候选。
S12	本拍没有请求。	剩余候选空闲，选中最老 A。

场景 06 一条命令连续仲裁失败

这里始终只有一条命令 A。它的目标半区在失败期间保持空闲，但下游一直没有给出完整的成功结果，所以 A 可以连续重试。



W06 VM VCD 20_retry_single

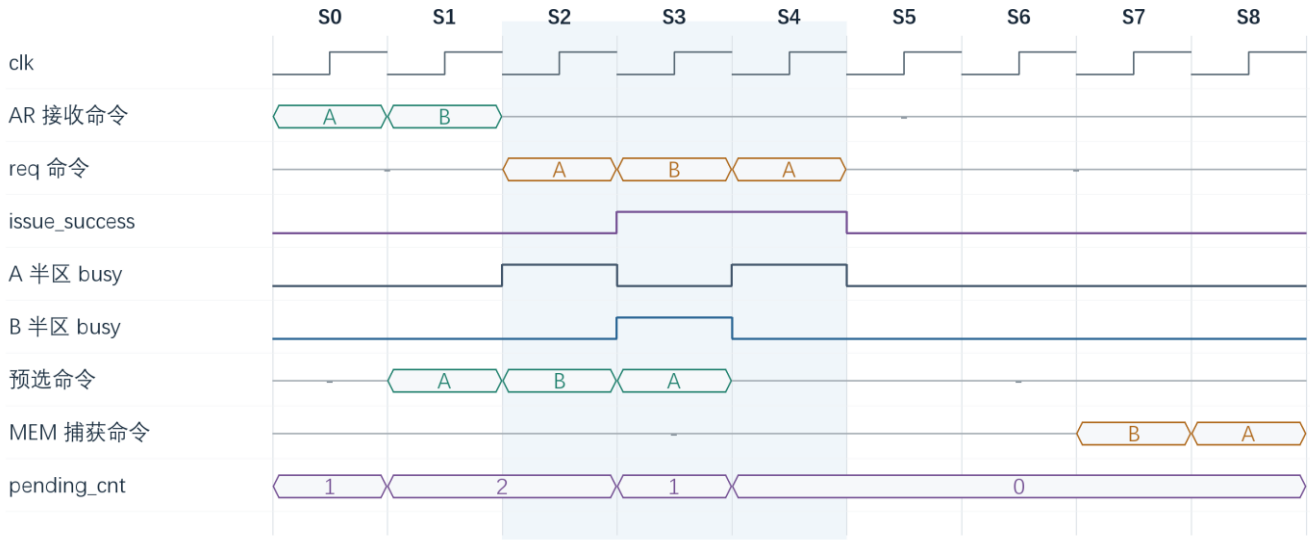
A 在 S0 接收、S1 选中，从 S2 开始请求 Bank3。S2 至 S6 虽然 ar_rd_gnt=FF，但 rw_rd_gnt=0，这几拍都不算成功。A 仍是窗口中最老的空闲候选，每拍重新被选中，所以 req 可以连续保持为 08。

S7 两个 grant 在 Bank3 位都为 1，A 才成功发出。该拍 Bank3 偶半区同时为忙，pending_cnt 变为零，A 不再参加选择。于是 S8 不再请求 A，S11 捕获它的 MEM 数据，S12 最早返回响应。

失败期间只是在尝试发送，不能为每一次尝试都安排一次 MEM 返回。OTB 只记录 S7 的成功事件，A 的年龄和命令内容在此前重试期间保持不变。

场景 07 失败后选择另一条命令

失败的命令不一定在下一拍继续请求。这个场景中，A 失败当拍的目标已经变忙，而窗口内的 B 可以访问，因此下一拍改为请求 B。



W07_VM_VCD_21_retry_busy_other

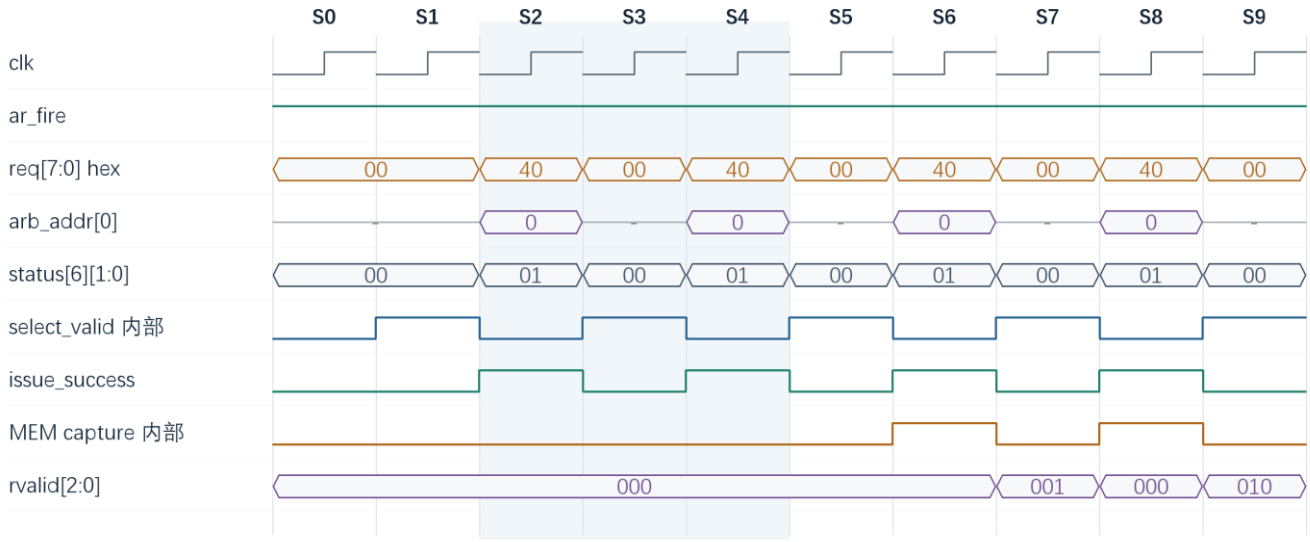
S2 请求 A 失败，选择逻辑仍保留 A，但它未通过空闲检查，所以选中 B。S3 请求 B 成功时，A 的目标重新空闲；A 又比其他候选更老，因此本拍重新选中 A，S4 请求 A 成功。

MEM 按成功发射的先后，在 S7 返回 B、S8 返回 A。若 A、B 来自同一 XAU，R 仍必须先返回 A，再返回 B。B 的数据虽然更早到达，仍要在 OTB 中等待。

是否连续重试原命令，由失败当拍重新计算的窗口、空闲状态和年龄共同决定。失败不清除命令，也不强制锁住发射流水只处理这一条。

场景 08 连续访问同一 Bank 半区

命令连续访问 Bank6 的偶半区，因此请求位为 req=40，arb_addr[0]=0。成功当拍目标变忙，会影响下一条命令的预选。



W08 VM VCD 25_same_half_pressure

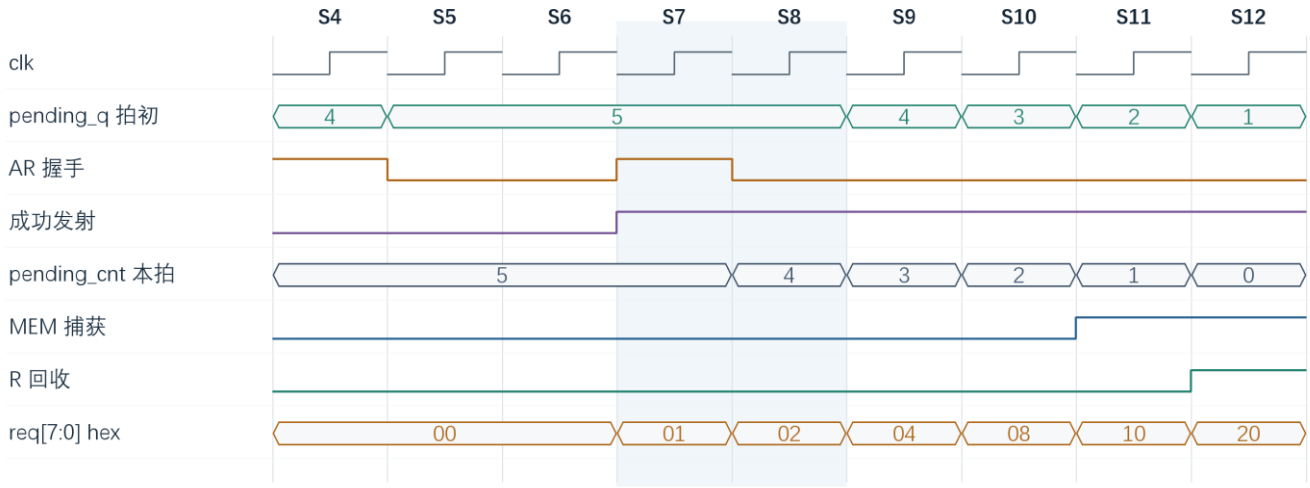
S2 成功时 rd_mem_status[6]=01，偶半区为忙，下一条访问相同半区的命令不能在 S2 被选中。S3 该半区重新空闲后才再次选择，因此下一次请求出现在 S4。这个空拍来自状态检查和流水时序，没有额外设置重试等待计数器。

rd_mem_status[b]	访问偶半区的命令	访问奇半区的命令
00	可以参加候选。	可以参加候选。
01	目标忙，排除。	可以参加候选。
10	可以参加候选。	目标忙，排除。
11	目标忙，排除。	目标忙，排除。

不能把两个状态位合成一个 Bank 忙信号使用。一个半区忙时，另一个半区仍可能有可访问命令；应按命令的 araddr[6] 选择相应状态位。

场景 09 待发计数立即反映本拍事件

这个场景检查 pending_cnt 的组合更新。图中的 pending_q 是寄存值，pending_cnt 是对外输出，两者在当前拍有接收或成功发射时可能不同。



W09 VM VCD 23_pending_delta

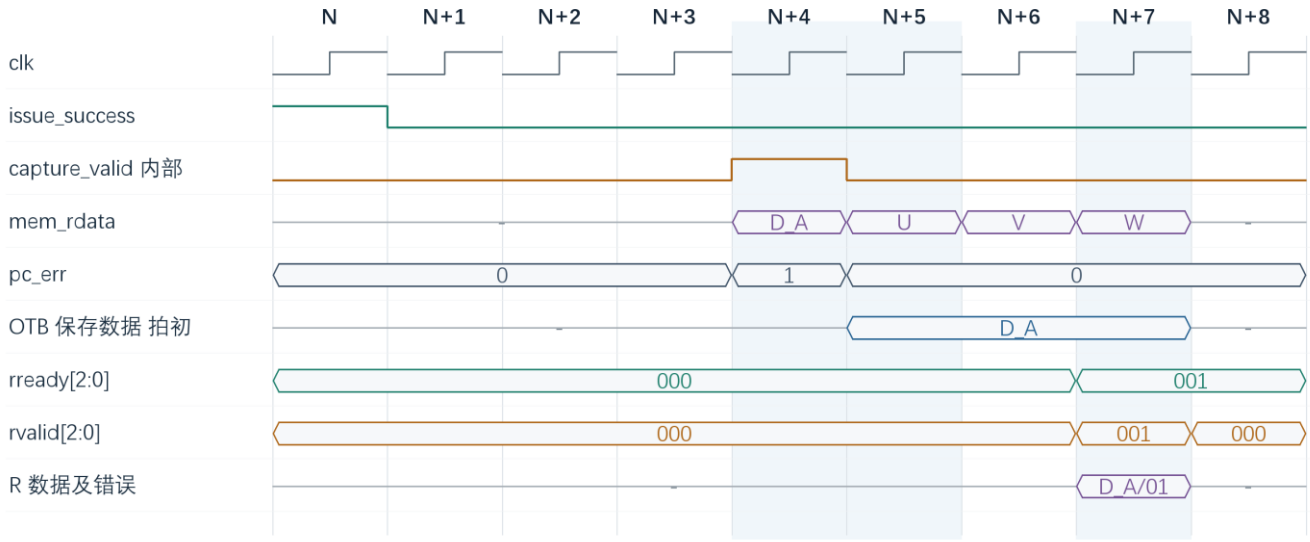
S7 拍初有五条待发命令，本拍接收一条新 AR，同时成功发射一条，所以 pending_cnt 仍为五。S8 没有新 AR，又成功发射一条，本拍输出立即变为四。

相同拍初值	本拍 AR 接收	本拍发射成功	本拍 pending_cnt
5	0	0	5
5	0	1	4
5	1	0	6
5	1	1	5

表中是从同一个拍初值出发的四种可能情况，不是四拍连续变化。仲裁失败不减计数；MEM 捕获和 R 回收也不直接改变待发数。R 可能释放容量，使新 AR 得以接收，这时增加计数的原因仍是 ar_fire。

场景 10 MEM 返回一拍而上游暂时不能接收

A 在 N 拍成功发射，但它的数据返回时，XAU0 还不能接收。OTB 必须先保存这份数据，不能要求 MEM 等待上游。



W10 协议示意：XAU0 队头 A，arresp=00；U/V/W 为非到期拍无关总线值

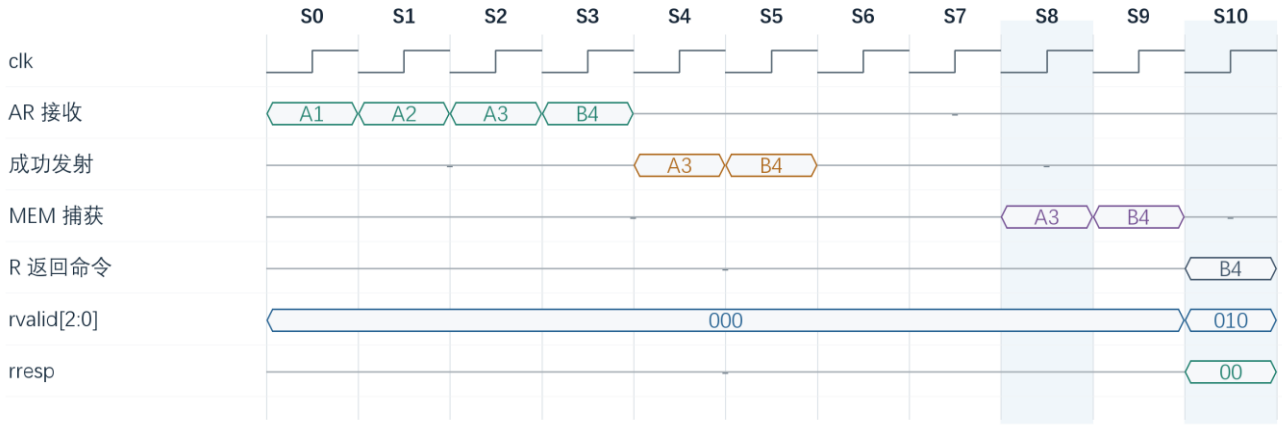
N+4 的 mem_rdata=D_A、pc_err=1 属于 A，OTB 在这一拍采样。后面的 U、V、W 是 MEM 总线上的其他值，不属于 A，不能覆盖已经保存的 D_A。

N+5、N+6 的 rready[0] 仍为 0，所以尽管 A 已完成，rvalid 也保持为 000。N+7 上游允许接收后，模块返回保存的 D_A 和 rresp=01，随后回收 A 的 entry。

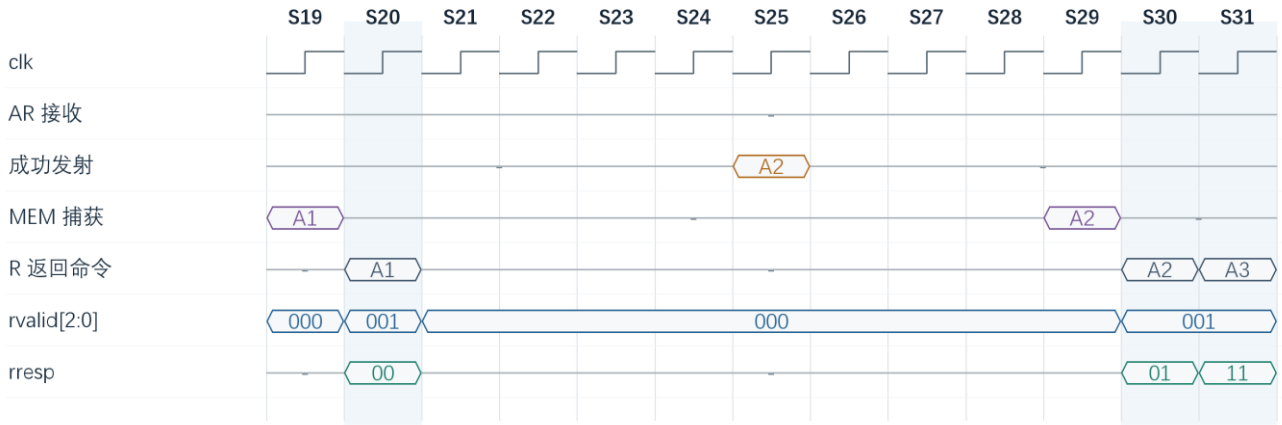
若 N+4 还存在另一条更早完成的命令，模块可以在捕获 A 的同时返回那条旧命令。限制仅在于 A 自己不能在 MEM 捕获当拍直接送到 R。返回计时流水也不能因为 rready 为零而暂停。

场景 11 第三条先完成仍然按序响应

XAU0 依次提交 A1、A2、A3，XAU1 另外提交 B4。A3 先访问成功并拿到数据，但 XAU0 必须按 A1、A2、A3 的顺序收到结果。



W11a VM VCD 28_xau0_order_312



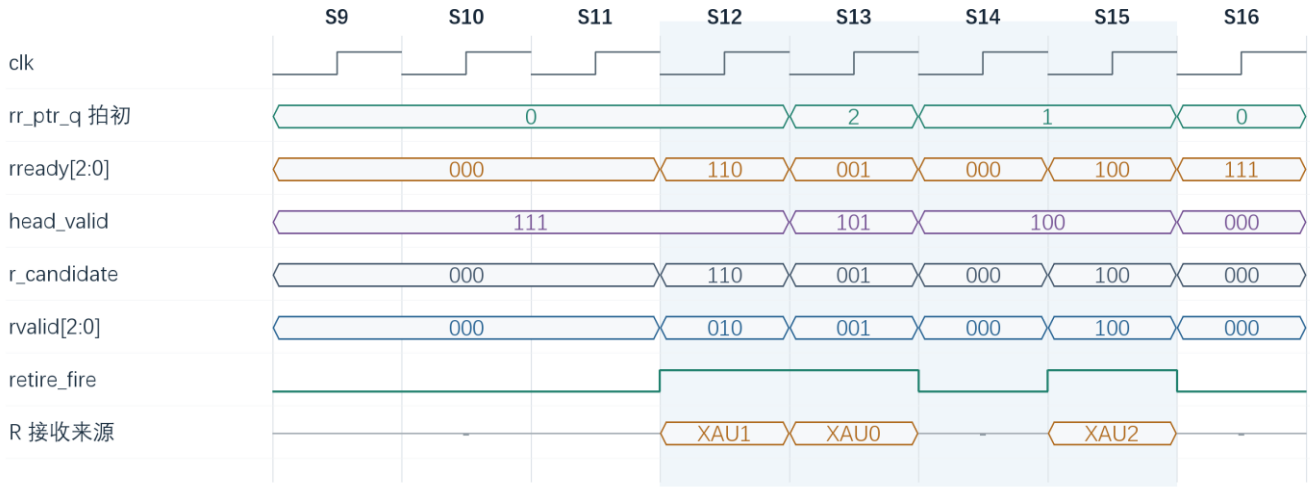
W11b VM VCD 28_xau0_order_312

A3 在 S8 已收到数据，仍需等待；XAU1 的 B4 可以在 S10 正常响应。XAU0 则依次在 S20、S30、S31 返回 A1、A2、A3。rready 全程为 111，所以等待完全来自保序要求。两图省略了 S11 至 S18，A1 在省略段的 S15 发射成功。

命令	AR 接收	发射成功	MEM 捕获	R 响应
A1	S0	S15	S19	S20
A2	S1	S25	S29	S30
A3	S2	S4	S8	S31
B4	S3	S5	S9	S10

场景 12 只在能够接收的来源之间轮询

三个 XAU 都可能有已经完成的队头，但 rready 会逐拍变化。轮询应跳过本拍不能接收的来源，并且只有实际响应后才推进指针。



W12 VM VCD 19_rr_ready_masks

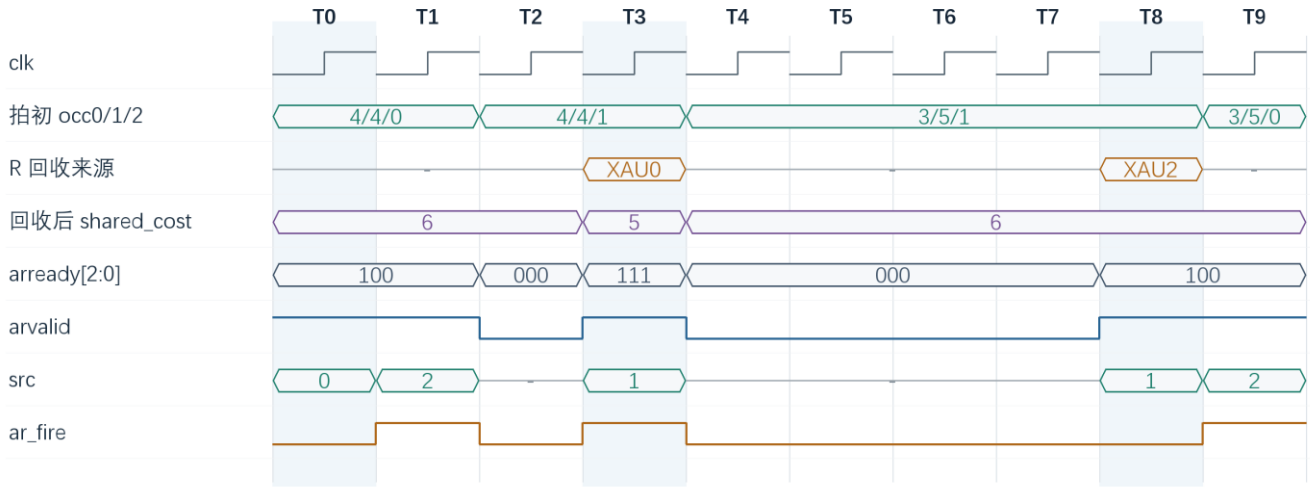
S9 至 S11 的 rready=000，没有响应，指针保持在 0。S12 变为 110，从 XAU0 开始扫描时跳过它，先选中 XAU1，响应后把下一次扫描起点改为 XAU2。

S13 只有 XAU0 ready，从 XAU2 循环扫描到 XAU0 后选中它，下一起点变为 XAU1。S14 再次没有 ready，指针保持；S15 只有 XAU2 ready，于是返回给 XAU2，指针回到 XAU0。

连续按 0、1、2 响应，只会出现在三个来源一直有完成队头、并且一直 ready 的情况下。轮询不会为了等待某个未就绪来源而闲置 R 通道，也不会在没有响应的拍自动移动指针。

场景 13 为第三个来源保留首条容量

这个场景说明为什么不能只检查每个 XAU 是否达到七项。XAU0 和 XAU1 各占四项时，虽然都未到七项，最后一组空间仍必须留给 XAU2。



W13 协议示意：T0/T8 的 src 不可接收；XAU2 的新事务最早在 T8 回收

T0 的占用为 4 / 4 / 0，六组共享容量已经用完，arready 只能为 100。上游给出的 XAU0 请求不被接收；T1 改为 XAU2 后接收，占用成为 4 / 4 / 1。

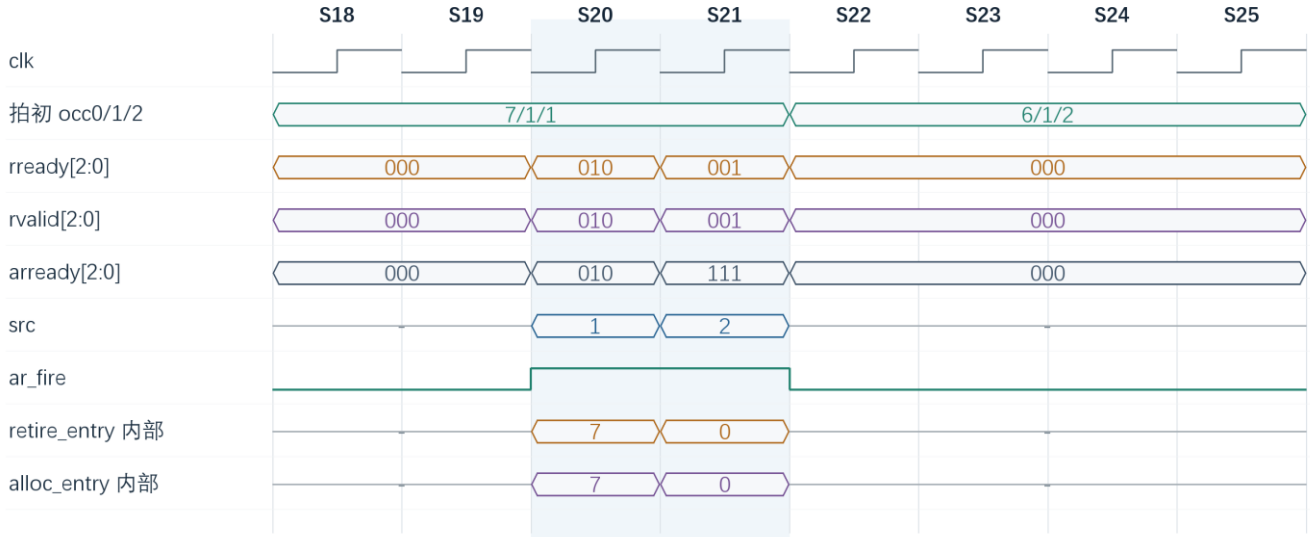
T3，之前已完成的 XAU0 队头响应，释放一组共享容量。这时三个来源都可以接收，上游选择了 XAU1，占用变成 3 / 5 / 1。T4 至 T7 没有回收，保持满池。

T8 回收的是 XAU2 唯一的一条命令，只能恢复 XAU2 的首条接收资格，因此 src=1 的新命令仍被拒绝。T9 上游改为 src=2 后才接收。这里释放一个 entry，不一定意味着三个来源都可以使用它。

XAU2 的旧命令在 T1 接收，最早 T2 选择、T3 成功、T7 捕获、T8 响应。图中保留了这条命令所需的完整时间，不能在接收后的任意下一拍就假设它可被回收。

场景 14 满容量时回收并接收新命令

九组 entry 全满时，本拍 R 响应释放的空间可以立即用于本拍 AR 接收，不需要额外等待一拍。



W14 VM VCD 10_capacity_reuse

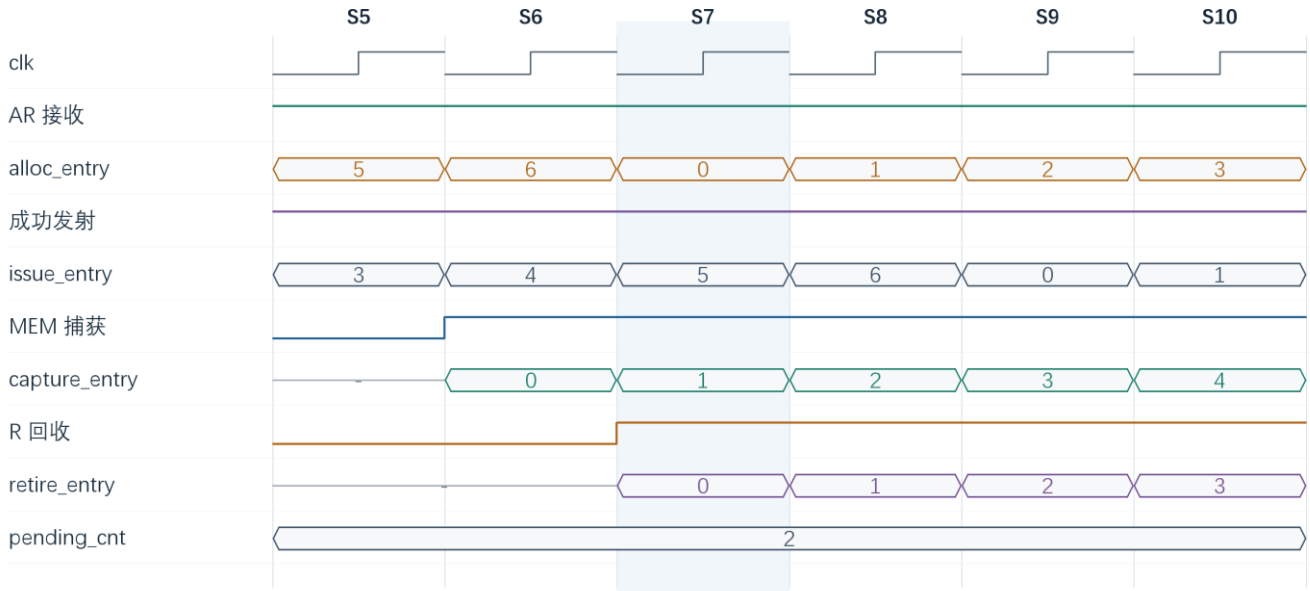
S18、S19 的占用为 7 / 1 / 1，且没有 R 响应，arready=000。S20 的 XAU1 唯一命令完成响应，恢复 XAU1 的接收资格，上游同拍以 src=1 提交新命令。

S20 的 retire_entry 和 alloc_entry 都是 7。采样沿前，R 仍读取编号 7 中的旧数据；采样沿后，编号 7 已经记录新命令，OTB 重新预留数据空间，完成和发射标志也被清零。

S21 回收 XAU0 的一条命令，释放的是共享容量，所以 arready=111。上游本拍选择 XAU2 接收后，占用变为 6 / 1 / 2。新命令必须成为最年轻项，不能因复用了旧编号而继承旧命令的优先级。

场景 15 接收发射捕获响应同拍发生

这一页重点看 S7。该拍既有新的 AR 接收，也有下游发射成功、MEM 数据捕获和 R 响应，用来说明四条处理路径如何同时更新。



W15 VM VCD 26_repeat_ids

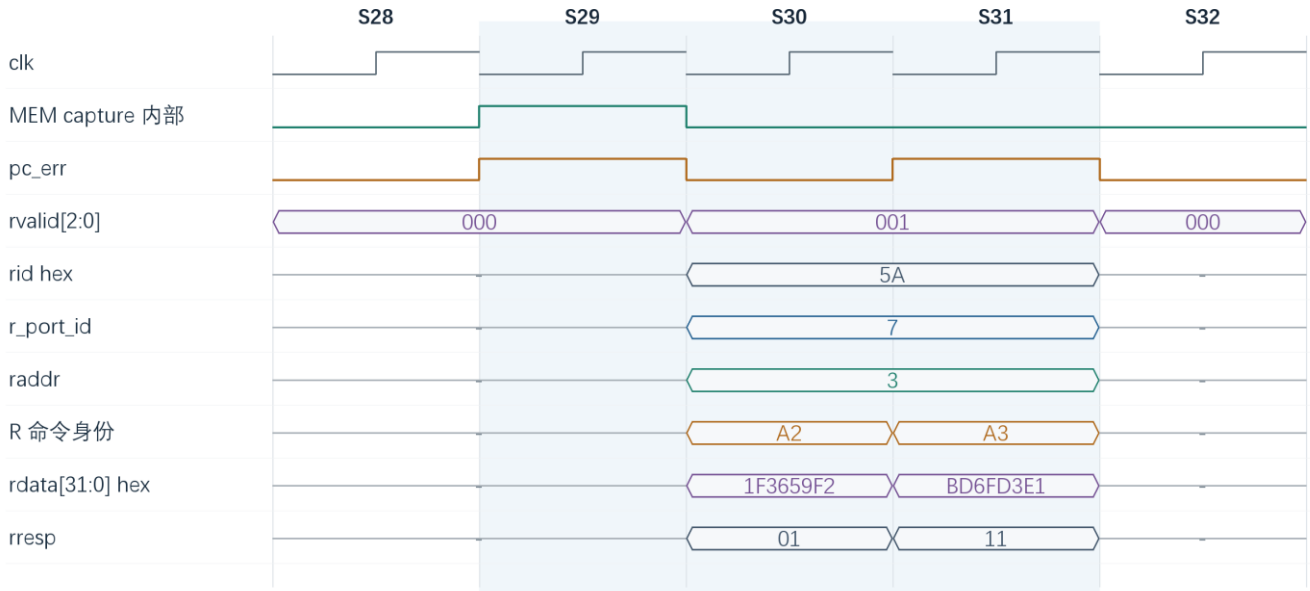
AR 为新命令保存字段；发射成功为已选命令记录返回时刻；MEM 为四拍前成功的命令保存数据；R 则交付更早完成命令的结果。图中的 entry 编号取自实际 RTL 记录，帮助区分这几件事各自作用于谁。

四种事件都必须完成，不能因为进入了 AR 处理分支就跳过 MEM 或 R。总占用只按 AR 增加、R 减少；待发数只按 AR 增加、成功发射减少。

若 AR 复用本拍 R 回收的编号，应按新命令重新初始化该项。MEM 捕获与 R 响应可以作用于不同 entry，但刚捕获的那一项不能同拍响应。同一 XAU 的接收和响应也可以同拍发生，留下的旧命令仍排在新命令之前。

场景 16 相同 ID 和不同错误结果

A1、A2、A3 的 ID 都为 5A，port_id 都为 7，raddr 都为 3。它们仍是三条独立命令，必须按接收顺序返回各自的数据和错误结果。



W16 VM VCD 31_xau0_order_312_repeat

命令	arresp	保存的 pc_err	响应拍	rresp
A1	00	0	S20	00
A2	00	1	S30	01
A3	11	1	S31	11

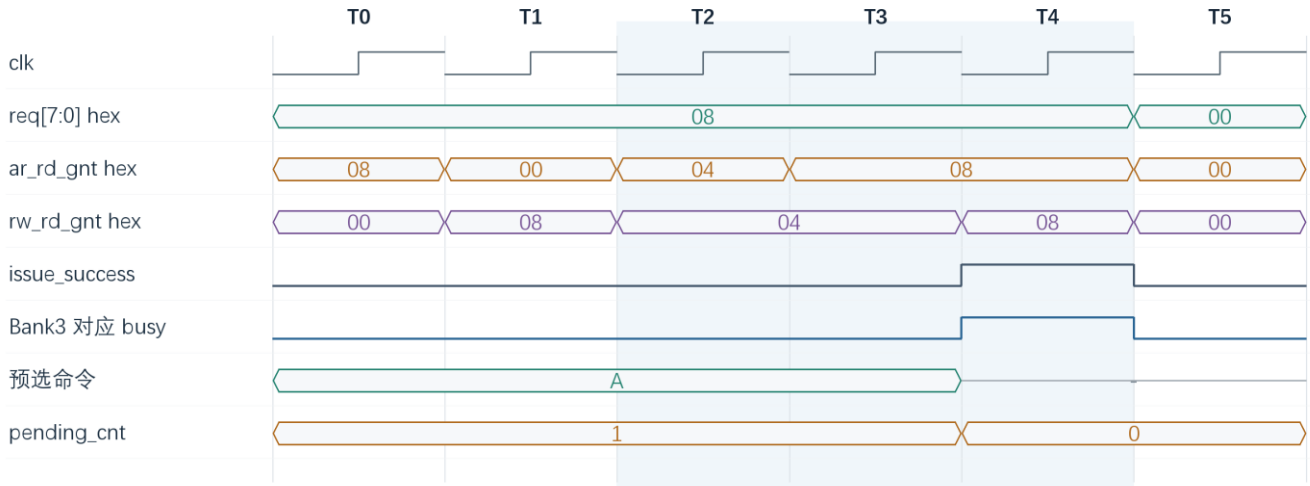
A3 在 S8 就已捕获数据，直到 S31 才能返回。它同时有地址越界和奇偶异常，按优先级返回 11。S31 当前 MEM 总线上的 pc_err 与这次响应无关，必须使用 A3 早先保存的结果。

响应命令	完整 128 位数据 十六进制
A1 S20	55AA00F8FE8698761234ABCD81012003
A2 S30	39249C0DFE8698766D7ED7D81F3659F2
A3 S31	8CB73912FE869876ECA053E7BD6FD3E1

判断顺序不能只看 ID。验证中要按 AR 接收次序记录每条命令，并逐项比较其地址、ID、port_id、完整数据和错误状态。

场景 17 两个仲裁结果必须匹配请求位

命令 A 持续请求 Bank3，req=08。图中给出几种容易误判的 grant 组合，只有最后一种表示 A 真正成功。



W17 协议示意：A 持续请求 Bank3，只有同一 bit 双 grant 才成功

T0 只有 ar_rd_gnt，T1 只有 rw_rd_gnt，都不成功。T2 两个结果虽然一致，但给的是 Bank2，不是当前请求的 Bank3。T3 两个结果分别位于不同 Bank，也不成功。

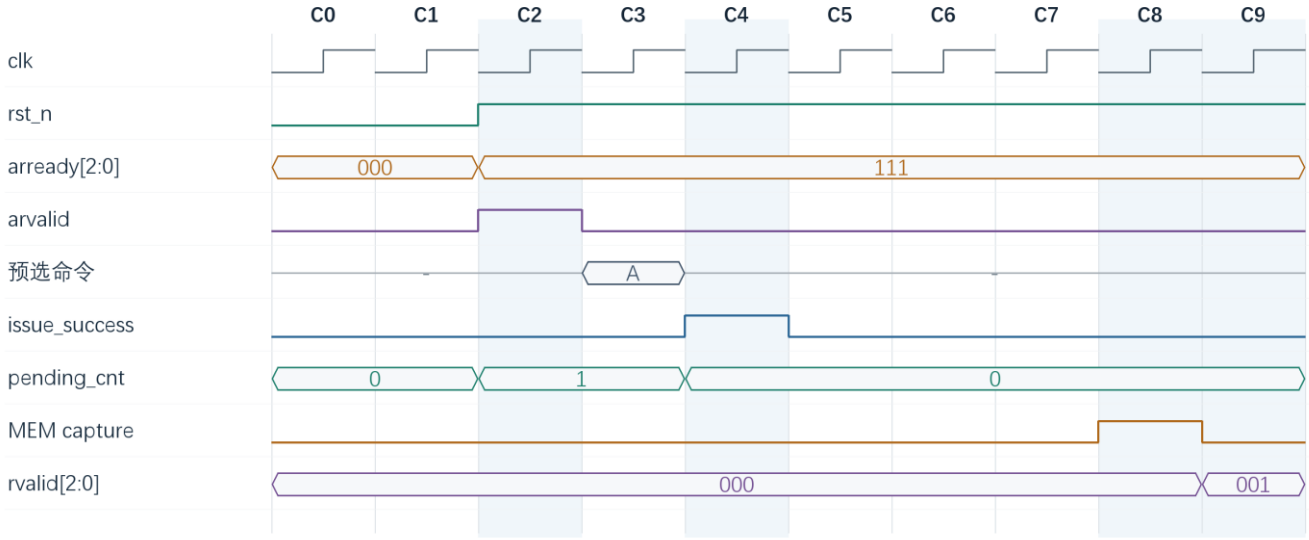
T4 的 req、ar_rd_gnt、rw_rd_gnt 在 Bank3 位同时为 1，才把 A 标记为已发射、减少待发计数，并让 OTB 开始计时。该拍目标半区状态同时为忙，T5 不再请求 A。

```
issue_success = |(req & ar_rd_gnt & rw_rd_gnt);
```

不能分别判断三个向量“是否有任意一位为 1”后再相与。grant 还可能包含其他 Bank 的位，本模块只使用与当前 req 匹配的位，不额外要求整个 grant 向量一定是独热。

场景 18 复位后接收第一条命令

本场景说明 RTL 从空状态启动时的行为，以及无效周期应该怎样理解。



W18 协议示意：空状态复位后启动；不定义带未完成 MEM 的单边中途复位

C0、C1 为低有效复位，命令记录、发射流水和返回计时全部清空。C2 释放复位并接收 A，C3 选择，C4 发射成功，C8 捕获 MEM 数据，C9 最早返回。复位前留下的流水内容不能生成新的请求。

req=0 时不解释 arb_addr，rvalid=0 时也不解释 R 的数据和标识。图中这些无效周期没有固定负载要求。当前 RTL 将部分无效输出清零，是实现行为，不是额外协议条件。

当前 SVA 统一使用 @(posedge clk) disable iff (!rst_n)，没有增加 started、active 或“第一拍复位后永久释放”的环境约束。运行中复位若涉及 MEM 中尚未结束的旧请求，需要系统另外约定如何清除或隔离旧返回。

8 验证内容与已有结果

验证围绕前面介绍的命令流程展开：接收时检查容量和来源，发射时检查窗口、年龄和仲裁结果，返回时检查数据保存、单 XAU 顺序及轮询。只连接顶层端口的检查器，通过独立记录已接收命令来判断输出是否正确。

需要检查的行为	对应检查入口
请求和响应只能选择一个对象	A_REQ_ONEHOT、A_R_ONEHOT、A_R_READY
输入来源合法及成功拍目标忙	E03_SRC、E04_ARRESP、E05_ZERO_READY、E06_SUCCESS_BUSY
容量保护和待发计数	A_READY、A_PENDING、A_PENDING_RECURRENCE
五项窗口及失败后重试	A_REQ、A_ARB_ADDR、C_RETRY_NEXT_CYCLE、C_NONHEAD_REFILL、C_BLOCKED_SIX
单 XAU 严格按序响应	每源独立 AR 接收 FIFO 及 A_XAU_R_* 属性
轮询结果与返回负载	A_RR_CHOICE、A_RR_BOUND、A_R_METADATA、A_R_DATA、A_R_RESP
同拍多事件和满容量复用	C_ALL_FOUR_EVENTS、C_FULL_REUSE、各源 SAME_CYCLE_POP_PUSH

目前有哪些证据

已有 VM 记录包含 11 个环境假设、40 个 DUT 断言、105 个参考模型自检实例和 46 个覆盖项。145 个断言实例通过 10 步 BMC；请求独热、响应独热和只向 ready 来源响应这三项取得无界证明。46 个覆盖项命中，33 个原生 SVA 仿真用例和 7 类保序注错回归通过。

有界检查、无界证明和覆盖命中代表不同范围，不能合并解释为全部功能已无界证明。真实 FormalMC 2024 执行及目标加密 RTL 等价检查，没有本轮完成结论。历史 Verilog 和早期验证包也不能代替当前 SystemVerilog 版本的验证。

1.2 版将状态口从 bit 改为 wire。VM 四态仿真逐位检查 0/1/X/Z，共 64 组输入、128 个端口观测值通过；此项检查接口传递。正常二值业务的 33 个原生 SVA 仿真用例及上述三组 SBY 任务已重新通过。

本文 11 张实际波形仍来自原始 VM VCD，另 8 张为协议示意；已核对其中 875 个采样周期。历史波形和注错结果保留原始来源，新增回归证据另存。

9 集成要求和文件入口

集成时应先使用 rtl/files.f 选择当前三份 .sv 文件，再按接口表连接上下游。同目录的 .v 是历史实现，不应同时编译。最需要核对的是 Bank 状态数组、12 位下游地址以及返回延迟。

上下游需要满足的条件

Arbit 在 req 有效的同拍给出两个仲裁结果，MEM 的实际接收时点与成功结果一致。成功拍目标半区的 rd_mem_status 位为忙，数据和 pc_err 在成功后的第四拍提供。若中间增加了寄存器，就需要重新检查这一时间关系。

上游按 src 对应的 arready 判断命令是否被接收。响应端只在 ready 的来源之间选择，因此上游不能依赖“先看到 rvalid 才拉高 rready”来启动接收，否则可能一直没有响应。对来源选择的公平性，需要由上游系统规定。

满容量同进同出和 pending_cnt 的组合更新会跨越模块接口。集成时应检查组合环、路径延迟和采样稳定性。运行中复位还需明确 MEM 在途命令怎样清除或隔离。

代码和证据的位置

仓库内路径	用途
rtl/files.f	当前 RTL 文件清单。
rtl/cmdbuf_rd.sv	顶层接口与子模块连接。
rtl/cmdbuf_rd_cmd.sv	容量、命令状态、年龄、窗口、发射和响应选择。
rtl/cmdbuf_rd_otb.sv	四拍返回计时、数据保存与回收。
formal/protocol_20261021/	当前端口 SVA、参考模型和运行入口。
verification/sva_readability_20261021/	引用的 VM 结果及原始 VCD 证据包。
verification/four_state_20261021/	四态接口修正及本次 VM 回归结果。
docs/timing/protocol_corrections_20261021.json	已确认的协议修正规则。
docs/design/rtl_detail_20261021/	本文图示、来源索引和文档核对记录。

本文基于代码 432ec88，并包含本次状态口 bit 改 wire 的修正。当前源文件校验值随新增验证证据保存。后续若更改协议或内部调度，应一起更新 RTL、参考模型、场景和验证结果。